

NETRA

Network Traffic Monitoring & Analysis Platform

Project Report

Course: Computer Networks

Author: Farah Tanveer

Institution: University of Karachi (UBIT)

Date: May 2026

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement & Objectives
4. Literature Review / Background Study
5. System Architecture & Design
6. Technology Stack
7. Module Descriptions
8. Implementation Details
9. Key Features
10. API Endpoints & Routes
11. Data Flow & Workflow
12. Testing & Results
13. Security Considerations
14. Future Enhancements
15. Conclusion
16. References

1. Abstract

Netra is a real-time network traffic monitoring and analysis platform developed as a semester project for the Computer Networks course. The platform captures live network packets using the Scapy library in Python, processes and logs them, and presents the data through a modern, responsive web dashboard built with Flask. It supports protocol-level filtering (TCP, UDP, ICMP), IP-based search, CSV data export, and comprehensive statistical analysis including protocol distribution, top source IPs, and destination port analytics. The system demonstrates fundamental networking concepts such as packet sniffing, protocol identification, port-to-service mapping, and real-time data streaming in a practical, user-friendly application.

2. Introduction

Computer networks form the backbone of modern digital communication. Understanding how data traverses a network at the packet level is essential for network administrators, security professionals, and students of computer science. Netra (meaning 'eye' in Sanskrit) is designed to provide real-time visibility into network traffic, enabling users to monitor, capture, analyze, and export packet data through an intuitive web-based interface.

The project bridges the gap between theoretical networking concepts taught in academia and their practical application. By implementing a live packet sniffer integrated with a full-stack web application, Netra demonstrates OSI/TCP-IP model concepts, protocol behavior, and traffic analysis in an accessible manner.

3. Problem Statement & Objectives

Problem Statement

Network traffic analysis tools like Wireshark are powerful but complex for beginners. There is a need for a lightweight, web-based, user-friendly platform that allows students and junior administrators to monitor and understand network traffic without steep learning curves.

Objectives

- Design and implement a real-time packet sniffing engine using Scapy
- Build a responsive web dashboard for live traffic visualization
- Provide multi-criteria filtering (Protocol, Source IP, Destination IP)
- Generate statistical analysis of captured traffic (protocol distribution, top sources, port analytics)
- Enable CSV export of captured data for offline analysis
- Implement session-based monitoring with persistent logging
- Demonstrate core Computer Networks concepts in a practical application

4. Literature Review / Background Study

4.1 Packet Sniffing

Packet sniffing is the practice of capturing data packets as they travel across a network. Tools like Wireshark, tcpdump, and Scapy enable this functionality. Scapy, a Python-based interactive packet manipulation library, provides granular control over packet capture and dissection, making it ideal for educational projects.

4.2 TCP/IP Protocol Suite

The TCP/IP model defines how data is packetized, addressed, transmitted, routed, and received. Key protocols relevant to this project include: TCP (Transmission Control Protocol) for reliable, connection-oriented communication; UDP (User Datagram Protocol) for lightweight, connectionless communication; ICMP (Internet Control Message Protocol) for diagnostic and error-reporting purposes; and IP (Internet Protocol) for addressing and routing.

4.3 Web-Based Monitoring Systems

Modern network monitoring solutions increasingly adopt web-based interfaces for accessibility. Tools like Nagios, Zabbix, and PRTG use dashboards for real-time visualization. Netra follows this paradigm using Flask as a lightweight web framework suitable for rapid prototyping and academic demonstration.

5. System Architecture & Design

5.1 Architecture Overview

Netra follows a three-tier architecture:

- 1. Presentation Layer - HTML/CSS/JS frontend with responsive UI
- 2. Application Layer - Flask web server handling routing, API endpoints, and business logic
- 3. Data Layer - CSV file storage, JSON stats persistence, and in-memory ring buffer

5.2 Directory Structure

```
Netra/
  app.py                # Main Flask application (287 lines)
  network_traffic.csv    # Captured packet data
  logs.txt              # System event logs
  stats.json            # Persistent session statistics
  utils/
    __init__.py          # Module exports
    packet_reader.py      # PacketSniffer class (229 lines)
  templates/
    index.html           # Landing page (174 lines)
    dashboard.html       # Monitoring dashboard (181 lines)
    stats.html           # Statistics page (148 lines)
  static/
    style.css            # Global stylesheet (305 lines)
    script.js            # Frontend logic (456 lines)
    images/              # Background assets
```

6. Technology Stack

Technology	Purpose
Python 3.13	Core programming language
Flask	Lightweight WSGI web framework
Scapy	Packet capture and dissection library
Pandas	Data manipulation and CSV analysis
HTML5 / CSS3	Semantic markup and premium styling
JavaScript (ES6+)	Frontend interactivity and async API calls
Flask-CORS	Cross-Origin Resource Sharing support

JSON	Statistics persistence format
CSV	Packet data storage format

7. Module Descriptions

7.1 PacketSniffer (utils/packet_reader.py)

The core engine of Netra. This module implements the PacketSniffer class which encapsulates all packet capture functionality:

- Thread-safe packet capture using Python threading and Scapy
- In-memory ring buffer (deque with maxlen=500) for real-time streaming
- Automatic CSV persistence for every captured packet
- Protocol identification (TCP/UDP/ICMP) with port extraction
- Port-to-service name mapping (35+ well-known services including HTTP, HTTPS, SSH, DNS, FTP, SMTP, MySQL, PostgreSQL, RDP, etc.)
- Session statistics tracking (packet count, byte count, protocol distribution)
- Start/Stop lifecycle management with clean state reset

7.2 Flask Application (app.py)

The application server provides 10 routes including 7 RESTful API endpoints. It manages the sniffer lifecycle, serves HTML templates, handles filtering logic using Pandas DataFrames, computes real-time statistics, and provides system logging with timestamps. Session tracking persists across restarts via stats.json.

7.3 Frontend (script.js + templates)

The frontend is a multi-page application with three views: Landing Page (marketing/intro), Dashboard (real-time monitoring), and Statistics (analytical insights). It uses vanilla JavaScript with async/await for API communication, implements a 1.5-second auto-refresh polling loop during active monitoring, and provides toast notifications, theme toggling (dark/light), scroll animations, and particle effects.

8. Implementation Details

8.1 Packet Capture Mechanism

Scapy's sniff() function is invoked in a daemon thread with a BPF filter 'ip' to capture only IP-layer traffic. The timeout parameter is set to 1 second, after which the sniff loop recursively restarts if monitoring is still active. This design ensures the sniffer remains responsive to stop commands while maintaining continuous capture. Each packet is dissected to extract: timestamp, source/destination IP, protocol type, source/destination ports, and packet size in bytes.

8.2 Thread Safety

A threading.Lock protects all shared state (packet_count, total_bytes, protocol_counts, packet_buffer). The ring buffer uses collections.deque with maxlen=500, providing O(1) append and automatic eviction of oldest entries, ensuring bounded memory usage during extended monitoring sessions.

8.3 Data Persistence

Captured packets are written to network_traffic.csv with columns: Time, Source_IP, Destination_IP, Protocol, Packet_Size, Source_Port, Destination_Port. The CSV is cleared at the start of each new session to provide clean

per-session data. Session metadata (total sessions, cumulative packets, cumulative data) persists in stats.json across application restarts.

8.4 Frontend Architecture

The UI uses a glassmorphism design system with CSS custom properties for theming. Key design patterns include: backdrop-filter blur effects, gradient text rendering, CSS Grid for responsive layouts, IntersectionObserver for scroll-triggered animations, and CSS keyframe animations for particle effects and UI micro-interactions.

9. Key Features

Real-Time Packet Capture:	Live sniffing with Scapy, 1.5s polling, instant UI updates
Protocol Filtering:	Filter by TCP, UDP, or ICMP in real-time
IP-Based Search:	Filter packets by source or destination IP address
Statistical Analysis:	Protocol distribution, top 5 source IPs, top 5 destination ports with percentages
CSV Export:	One-click download of captured traffic data
Dark/Light Theme:	Persistent theme preference with Ctrl+K keyboard shortcut
System Logging:	Timestamped event logs for all monitoring actions
Responsive Design:	Mobile-first layout adapting to all screen sizes
Service Identification:	Automatic port-to-service mapping (HTTP, DNS, SSH, etc.)
Session Management:	Clean state reset per session with cumulative statistics

10. API Endpoints & Routes

Endpoint	Description
GET /	Landing page (index.html)
GET /dashboard	Dashboard page (dashboard.html)
GET /stats	Statistics page (stats.html)
GET /api/status	Returns monitoring status and live stats
POST /api/start-monitoring	Starts packet capture, clears CSV, increments session count
POST /api/stop-monitoring	Stops capture, persists cumulative stats
GET /api/live-packets	Returns real-time packets from ring buffer with filters
GET /api/traffic-data	Returns filtered historical data from CSV via Pandas
GET /api/stats	Returns detailed statistics (protocol dist, top IPs, top ports)
GET /api/logs	Returns recent system log entries
GET /api/export-csv	Downloads network_traffic.csv as attachment

11. Data Flow & Workflow

1. User clicks 'Start Capturing' on the Dashboard
2. Frontend sends POST /api/start-monitoring
3. Backend initializes PacketSniffer: clears CSV, resets counters, starts daemon thread
4. Scapy captures IP packets on the default interface
5. Each packet is dissected, appended to ring buffer, and written to CSV

6. Frontend polls GET /api/live-packets every 1.5 seconds
7. Dashboard renders packet table, updates stat counters in real-time
8. User clicks 'Stop' - POST /api/stop-monitoring
9. Backend stops sniffer thread, persists cumulative stats to stats.json
10. User navigates to Statistics page for deep analysis
11. Frontend fetches GET /api/stats - backend computes analytics via Pandas
12. User can export captured data as CSV at any time

12. Testing & Results

The application was tested on Windows 10/11 with Python 3.13 and Npcap driver for packet capture. Testing verified:

- Successful capture of TCP, UDP, and ICMP packets on local network
- Correct protocol identification and port extraction
- Accurate port-to-service mapping for well-known services
- Real-time UI updates during active monitoring sessions
- Proper CSV file generation with valid data formatting
- Filter functionality for protocol, source IP, and destination IP
- Statistics computation accuracy (percentages, averages, counts)
- Theme persistence across page navigation and browser sessions
- Clean session reset (CSV cleared, counters zeroed) on new capture
- Responsive layout on desktop and mobile viewports
- System log accuracy with proper timestamping

Sample test session captured 49 packets with 19.34 KB of data, correctly identifying TCP, UDP, and ICMP protocol distributions with accurate service name resolution.

13. Security Considerations

- Packet capture requires administrator/root privileges (Npcap on Windows)
- CORS is enabled via Flask-CORS for development flexibility
- Input validation on API query parameters prevents injection
- The application runs on localhost (127.0.0.1) by default for safety
- No sensitive packet payload data is stored - only metadata headers
- CSV and log files use UTF-8 encoding to prevent encoding attacks

14. Future Enhancements

- Deep Packet Inspection (DPI) for payload-level analysis
- Integration with machine learning models for anomaly/intrusion detection
- Database backend (SQLite/PostgreSQL) replacing CSV for scalability
- WebSocket-based real-time streaming replacing polling
- PCAP file import/export for compatibility with Wireshark
- User authentication and role-based access control
- Network topology visualization and geographic IP mapping

- Bandwidth monitoring and alerting thresholds
- Docker containerization for easy deployment

15. Conclusion

Netra successfully demonstrates the practical application of computer networking concepts through a fully functional, real-time network traffic monitoring platform. The project integrates packet sniffing (Scapy), web development (Flask), data analysis (Pandas), and modern UI design into a cohesive system that captures, processes, visualizes, and exports network traffic data. The platform provides an accessible alternative to complex tools like Wireshark for educational purposes, while maintaining extensibility for future enhancements. Through this project, core concepts of the TCP/IP protocol suite, packet structure, port-service associations, and network analysis have been explored and implemented in a hands-on manner.

16. References

- [1] Kurose, J.F. & Ross, K.W. - Computer Networking: A Top-Down Approach, 8th Edition
- [2] Scapy Documentation - <https://scapy.readthedocs.io/>
- [3] Flask Documentation - <https://flask.palletsprojects.com/>
- [4] Pandas Documentation - <https://pandas.pydata.org/docs/>
- [5] RFC 793 - Transmission Control Protocol (TCP)
- [6] RFC 768 - User Datagram Protocol (UDP)
- [7] RFC 792 - Internet Control Message Protocol (ICMP)
- [8] MDN Web Docs - <https://developer.mozilla.org/>
- [9] Npcap Documentation - <https://npcap.com/>